

COSC3060. Web Programming Studio

Project Title

Assignment 2. Web Application Prototype

Date of submission

Name of your team: SGS1-G6

Team member	Student ID	Module
Đỗ Đức Lộc	S4178227	"Blog"
Dương Quang Tâm	S4197086	"Discussion Forum"
Nguyễn Hữu Khôi	s4162137	"Shopping cart", "User Account Management"
Nguyễn Anh Khôi	s4216408	"Product review and rating", "Shared module: Sitemap"

I declare that in submitting all work for this assessment, I have read, understood, and agree to the content and expectations of the assessment declaration.

1. Proposed product

1.1. Brief description

Our product is a marketplace-style web application that helps customers/ project creators present their ideas and receive support from other users or certified makers.

The main users are project creators, makers, and administrators. Creators can publish project requests, negotiate prices for the listings, post updates, and interact with supporters. Makers can discover projects, make purchases to reserve a project to work on through the shopping cart, join discussion forums, and submit reviews.

Administrators manage users and moderate platform content. The application includes Shopping Cart, Discussion Forum, Blog, Product Review and Rating, User Account Management, Administration, and Sitemap modules. Together, these modules support the workflow from project discovery to pledge confirmation and community interaction.

1.2. Database schemas

Database Schema Overview

The project uses a lightweight JSON-based database stored in `db.json`, managed by `backend/src/data/db.js` and accessed through the Express API in `server.js`, rather than a traditional relational database. The `users` collection supports authentication, registration, profiles, roles, account status, contact details, profile pictures, and notification preferences; `carts` stores session-based shopping carts containing item IDs, quantities, prices, and descriptive details; and `orders` records completed checkouts with order numbers, session references, copied cart items, payment details, delivery information, subtotals, taxes, shipping costs, totals, and timestamps. The Discussion Forum module uses threads and replies, where each thread stores its author, title, content, image URL, status, posting date, and reply count, while replies reference their parent thread and store author, title, content, proposed price, image URL, and timestamp; uploaded forum images are saved under the backend uploads directory. The Marketplace module builds listings from forum threads and replies, while the Wishlist module currently persists saved marketplace threads in browser `localStorage` rather than the active JSON database. Similarly, the Reviews and Ratings module currently stores reviews, filters, and drafts in `localStorage`, and the Blog module primarily uses static HTML content and image URLs, although the project change notes describe planned or branch-specific backend collections for blogs, reviews, and wishlists. The Homepage and Sitemap modules do not introduce business data: the homepage presents links and summaries, while the sitemap stores generated page and link metadata. The Admin module operates on the shared `users` collection for user search, role management, activation/deactivation, deletion, and account administration.

The main data collections include:

Users

Stores account information including user ID, first name, last name, username, email address, password, contact information, profile description, profile picture, account status, and preferences.

User Preferences

Stores notification settings for each user, including email notifications, message notifications, and new-request notifications. These are embedded within the user record.

Carts

Stores temporary items selected before checkout. Carts are currently session-based, using a sessionID, and each cart can contain multiple cart items.

Cart Items

Stores individual items within a cart. Each item contains a product or forum-thread reference, quantity, unit price, and extra display details such as the item title, maker, and accepted-offer status.

Orders

Stores confirmed checkout records. An order includes the session cart that created it, purchased items, payment details, delivery details, subtotal, service cost, tax, final total, order number, and creation date.

Order Items

Stores a snapshot of each cart item at the time of checkout. This preserves the original product reference, quantity, unit price, and relevant item details even after the cart is cleared.

Payment Details

Stores limited payment information needed for the prototype, including cardholder name, card brand, and final four card digits. Full card details are not stored.

Delivery Details

Stores delivery information for an order, including recipient name, phone number, address, city, state, postal code, country, and optional delivery instructions.

Discussion Threads

Stores custom-maker requests posted by users. A thread includes the title, description, author, creation date, reference image, request status, and reply count.

Discussion Replies

Stores replies and price offers posted under a discussion thread. Each reply is linked to one thread and includes its author, title, content, optional price, image, and posting date.

Blog Posts:

Stores articles published through the Blog module. Each blog post includes a blog ID, author, title, publication date, category, tags, image URL, summary, and full article content. The author can be associated with a user through `authorId`, and the blog record supports ownership checks

for editing or deleting posts.

Reviews and Ratings:

Stores product feedback submitted through the Review module. Each review includes a review ID, reviewer, product reference, product name, title, category, numeric rating, summary, detailed description, image, and date added. The reviewer can be associated with a user through 'reviewerId', while 'productId' is a logical reference to the reviewed marketplace item. Review ownership is used when users edit or delete their own reviews.

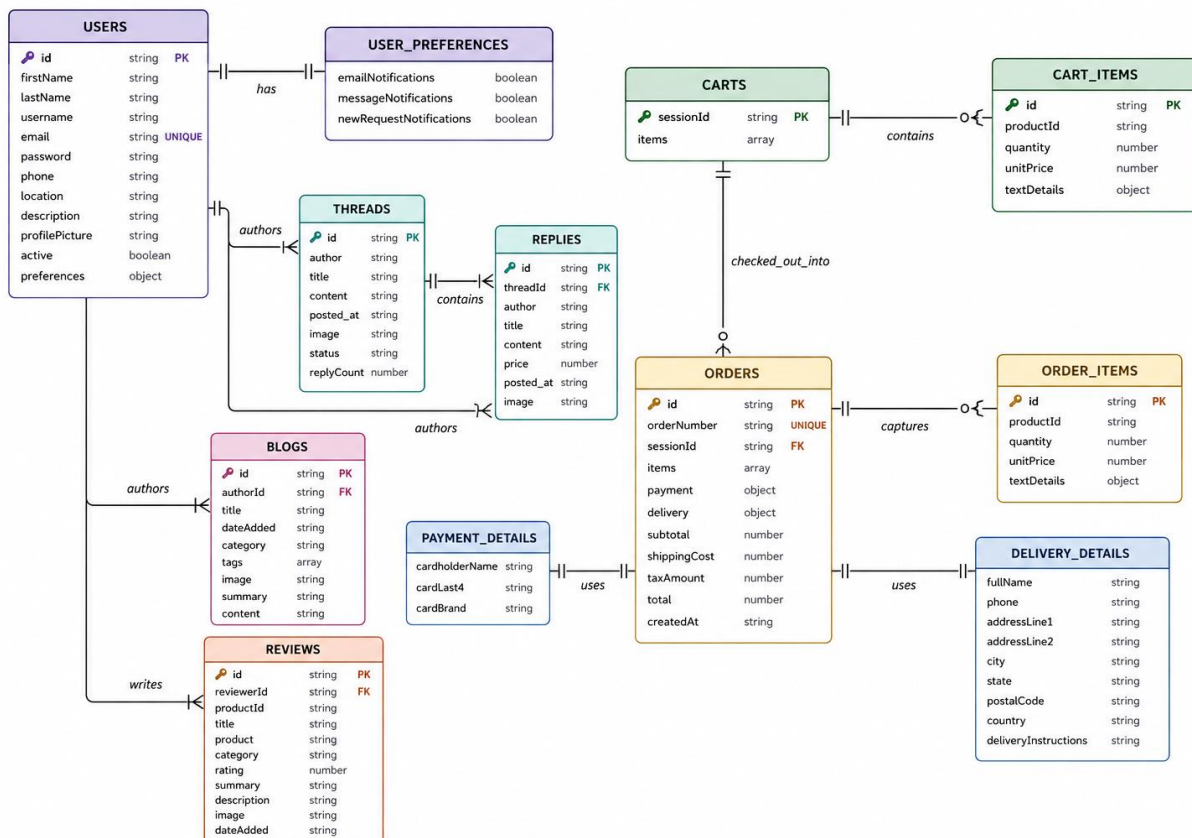


Figure 1. Database schema diagram generated by ChatGPT based on description

2. Individual module (Individual Work)

Individual Module: Shopping Cart (Huu Khoi)

This section describes the Marketplace and Shopping Cart module and the shared User Account functionality it uses. Users first open the marketplace to browse custom-maker requests retrieved from the backend. They can search, filter, and sort those requests in the browser, open the related discussion thread, and add a request with an agreed price to a session cart. The cart is then loaded from the backend, where users can remove items before proceeding to checkout. Submitting checkout creates a persisted order and clears the session cart. The cart is currently linked to a browser sessionId, which is stored in Web Storage and sent with each cart request. User Account

provides registration, login, profile, and account-management endpoints for the shared module. Authentication is not yet enforced as a server-side requirement for cart actions in this current prototype.

Table 1. Routes of Shopping cart module and Project module

Feature	Description	Method	URL / Endpoint	Module
View marketplace requests	Retrieves custom-maker requests, including their status, reply count, and latest price offer.	GET	shopping_cart/products.html and /api/marketplace	Marketplace / Shopping Cart
Search, filter, and sort requests	Filters the loaded marketplace data by text and status, then sorts it by offers, agreed price, or title without a backend request. Filter selections persist after refresh.	Client-side JavaScript	shopping_cart/products.html	Marketplace / Shopping Cart
View request discussion	Opens the forum thread that provides the request details and price offers.	GET	discuss_forum/thread-detail.html?threadId=:threadId and /api/threads/:id	Shared Discussion Forum
Add agreed offer to cart	Creates a backend cart item for a	POST	/api/cart/:sessionId/items	Shopping Cart

Feature	Description	Method	URL / Endpoint	Module
	marketplace request with a valid agreed price.			
View cart	Retrieves all items and the subtotal for the current browser session.	GET	shopping_cart/cart.html and /api/cart/:sessionId	Shopping Cart
Update cart item	Changes a cart item's quantity or stored item details.	PUT	/api/cart/:sessionId/items/:itemId	Shopping Cart
Remove cart item	Deletes a selected item from the current session cart.	DELETE	/api/cart/:sessionId/items/:itemId	Shopping Cart
Checkout	Validates delivery and payment details, creates an order from the cart, and clears the cart after success.	POST	shopping_cart/checkout.html and /api/orders/checkout/:sessionId	Shopping Cart
View confirmation	Displays the order confirmation page after checkout.	GET	shopping_cart/order-confirmation.html?orderNumber=:orderNumber	Shopping Cart
Register account	Creates a user account for the shared	POST	user_account/auth.html and /api/auth/register	Shared User Account

Feature	Description	Method	URL / Endpoint	Module
	User Account module.			
Log in	Authenticates an existing user and stores the returned user session details in Web Storage.	POST	user_account/auth.html and /api/auth/login	Shared User Account
View or edit profile	Retrieves or updates account profile information.	GET, PUT	user_account/profile.html and /api/auth/user/:id	Shared User Account

Individual Module: Discussion_forum (Tam Duong)

This section lists the application's features and the API endpoints used to access them, covering both the individual module I developed independently (Discussion Forum) and the shared modules used collaboratively with my team.

In the Discussion Forum module, a logged-in user first views the request board, where all open custom-make requests are listed and can be searched by title or content, and sorted using Date (oldest, newest) or Title(A-Z or Z-A) through *GET /api/threads*. Selecting a request opens its detail page via *GET /api/threads/:id*, which returns the thread together with all of its information and you can post a reply on that thread. A user can create a new request through *POST /api/threads* (with an optional reference image), and other users can respond with a reply or a price offer which will change the status of the thread on the forum through *POST /api/threads/:id/replies*. Only the original author of a thread or reply can edit it (*PUT /api/threads/:id*, *PUT /api/replies/:replyId*) or delete it (*DELETE /api/threads/:id*, *DELETE /api/replies/:replyId*); deleting a thread also cascades to delete all of its replies. Every create, edit, and delete action is protected on the server by an authentication check and an ownership check, so a request cannot be spoofed into modifying another user's post.

This module also relies on two shared modules developed with my team. First, the Shared User Account module: only users authenticated with a bearer token (*POST /api/auth/login*, *POST /api/auth/register*) may create threads, post replies, or edit/delete their own posts, and the server resolves the author from this token rather than trusting any value sent by the client. Second, the Shopping Cart module: when a buyer agrees to a maker's proposed price on a reply, the item is added to the shared cart through *POST /api/cart/:sessionId/items* and the corresponding thread is updated to "Closed" status, connecting the forum's negotiation flow to the checkout flow.

Feature	Description	Method	URL / Endpoint	Module
View request board	Displays all custom-make requests (threads). Supports live search by title/content and sorting (newest, oldest, title A-Z/Z-A).	GET	Discuss_forum/forum.html and /api/threads	Discussion Forum

View request details	Displays the full content of a single request thread together with all of its replies/offers.	GET	Discuss_forum/thread-detail.html and api/threads/:id	Discussion Forum
Create request	Allows a logged-in user to post a new request thread with a title, description and a reference image.	POST	Discuss_forum/create-thread.html and /api/threads	Discussion Forum
Edit request	Allows the thread's original author to update its title, content, or reference image.	PUT	Discuss_forum/edit-post.html and /api/threads/:id	Discussion Forum
Delete request	Allows the thread's original author to delete the request; all of its replies are cascade-deleted as well.	DELETE	/api/threads/:id	Discussion Forum
Post reply / offer	Allows a logged-in user to reply to a request, optionally proposing a price and attaching an image.	POST	/api/threads/:id/replies	Discussion Forum
Edit reply	Allows the reply's original author to update its title, content, price, or image.	PUT	/api/replies/:replyId	Discussion Forum
Delete reply	Allows the reply's original author to delete their own reply and decrements the thread's reply count.	DELETE	Discuss_forum/edit-reply and /api/replies/:replyId	Discussion Forum
Agree price & add to cart	Adds an agreed-upon offer to the buyer's shopping cart as a cart item and marks the related thread as "Closed."	POST	/api/cart/:sessionId/items	Shared / Shopping Cart
Log in	Authenticates an existing user and issues a bearer token used to identify the author of new threads/replies.	POST	/api/auth/login	Shared User Account
Register account	Allows a new user to create an account before posting requests or replies.	POST	/api/auth/register	Shared User Account

Individual Module: Product Review and Rating (Nguyen Anh Khoi)

The Product Review and Rating module allows users to view, search, filter, sort, create, edit, and delete product reviews. Each review contains information such as reviewer name, product reference, product name, review title, category, numeric rating, summary, detailed description, optional image, and date added. The module can also associate a review with a registered user through `reviewerId` and with a marketplace product through `productId`, which supports review ownership and connection with other modules.

Users can browse existing reviews and use search, rating filters, category filters, and sorting options to find relevant feedback. When submitting a review, client-side validation checks the required fields and rating before the review is saved. Users can also edit or delete reviews that belong to them. In the current prototype, reviews, filter selections, and unfinished review drafts are stored using the browser **Web Storage API (localStorage)**, allowing the data to remain available after refreshing the page.

The module is connected to the **Marketplace** through the reviewed product reference and to the **User Account module** through reviewer information. This allows reviews to identify both the product being reviewed and the user who submitted the feedback.

Feature	Description	Implementation
---------	-------------	----------------

View Reviews	Displays product reviews and rating information.	JavaScript
Search / Filter / Sort	Finds reviews by keyword, rating, category, or sorting option.	JavaScript
Submit Review	Validates and saves a new rating and review.	JavaScript / localStorage
Edit Review	Allows users to update their own reviews.	JavaScript / localStorage
Delete Review	Allows users to remove their own reviews.	JavaScript / localStorage
Save Draft	Stores unfinished review form data for later use.	Web Storage API

Individual Module: Blog (Do Dac Loc)

1. Module Description & Features The Blog Module provides an engaging platform for users to read, filter, search, and manage articles related to products, tech news, and user guides.

Blog Listing & Categorization: Displays a structured grid of blog posts categorized by topics (e.g., News, Tech Reviews, Tutorials) with filtering and pagination.

Featured Posts Banner: Highlights top-trending or sticky articles at the top of the blog homepage.

Search & Filter System: Allows users to dynamically search posts by keywords or filter by tags/categories without reloading the page.

Blog Detail Page: Displays rich content including headers, embedded images, publication date, author details, and a dynamic comment/discussion section.

Draft & Bookmark Feature: Enables logged-in users to save articles for offline reading or bookmark posts using localStorage.

2. Key Technical Contributions & Features Table

Feature	Description	Implementation Strategy / Storage
Search & Tag Filter	Filters blog posts instantly as the user types or selects a tag.	JavaScript DOM Manipulation & Array filtering
Bookmark / Save Post	Saves favorite posts to the user's reading list.	Web Storage API (localStorage)
Read Time Estimator	Automatically calculates and displays estimated reading time based on word count.	JavaScript text length calculations
Interactive Comments	Allows users to view and submit comments on blog posts.	JavaScript dynamic element rendering

3. HTML / CSS / JS File Structure

Static_html_css/Blog/blog.html: Main blog homepage featuring the hero banner, category filters, and article cards.

Static_html_css/Blog/blog-detail.html: Individual post view showing article body, author metadata, and comments section.

css/blog.css: Responsive styling for blog grid layout, typography, featured cards, and comment styling.

js/blog.js: Handles search query filtering, bookmark toggling via localStorage, and comment submission logic.

4. UI/UX Design & Implementation

Grid System: Responsive CSS Grid and Flexbox layout ensuring optimal viewing across mobile, tablet, and desktop screens.

Visual Hierarchy: Clean typography hierarchy with clear headers, high-contrast text, and generous spacing to improve readability.

Interactive Elements: Subtle hover animations on post cards, active state highlights for category filters, and bookmark icon state toggling.

5. Self-Evaluation & Reflection

Achievements: Successfully implemented a responsive and clean blogging interface with dynamic client-side filtering and storage management.

Challenges & Solutions: Managing state updates for saved articles across multiple pages was resolved using unified key formatting in localStorage.

Future Improvements: Plan to integrate backend API endpoints for real-time article fetching, pagination, and persistent comment databases.

References (APA 7th Referencing style)

